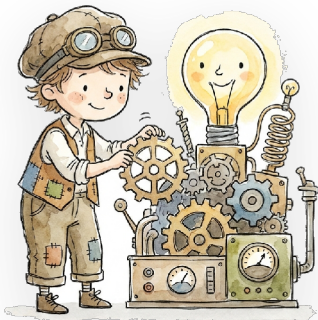




PROBABILITY & STATISTICS

The Naive-Bayes Classifier



YOUR GUIDE

Let's get started.

We can begin by examining a characteristic of one of the most reliable classifiers used in production environments: it operates by making an assumption that is rarely correct in practice. The **Naive-Bayes classifier** evaluates an email and determines whether the presence of the word "free" and the word "prize" provide information *jointly*, but it ultimately processes them as completely separate events. It treats these words as if they occurred independently, similar to two unrelated items appearing in the same inbox. This assumption is incorrect, however, because "free" and "prize" frequently appear together in spam messages. Despite this

limitation, the classifier built on this premise has successfully filtered junk mail for decades and remains the standard baseline that we must evaluate other models against.

In the [previous chapter](#) we built **Bayes' theorem** as a rule for updating a belief once evidence arrives. This chapter turns that one-line rule into a machine that takes a document and returns a label. The whole trick is a single deliberate simplification, and by the end you will see exactly why it is both a lie and a bargain.

The puzzle: Bayes with more than one clue

Recall the shape of Bayes' theorem. For a class **c** and some observed evidence, the **posterior** — our belief in the class *after* seeing the evidence — is

$$P(c|\text{evidence}) = P(\text{evidence}|c) \cdot P(c)/P(\text{evidence})$$

where **P(c)** is the **prior** (how common the class is before we look),
P(evidence | c) is the **likelihood** (how well the class explains what we

saw), and $P(\text{evidence})$ is the **evidence**, a normalizing constant that is the same for every class.¹

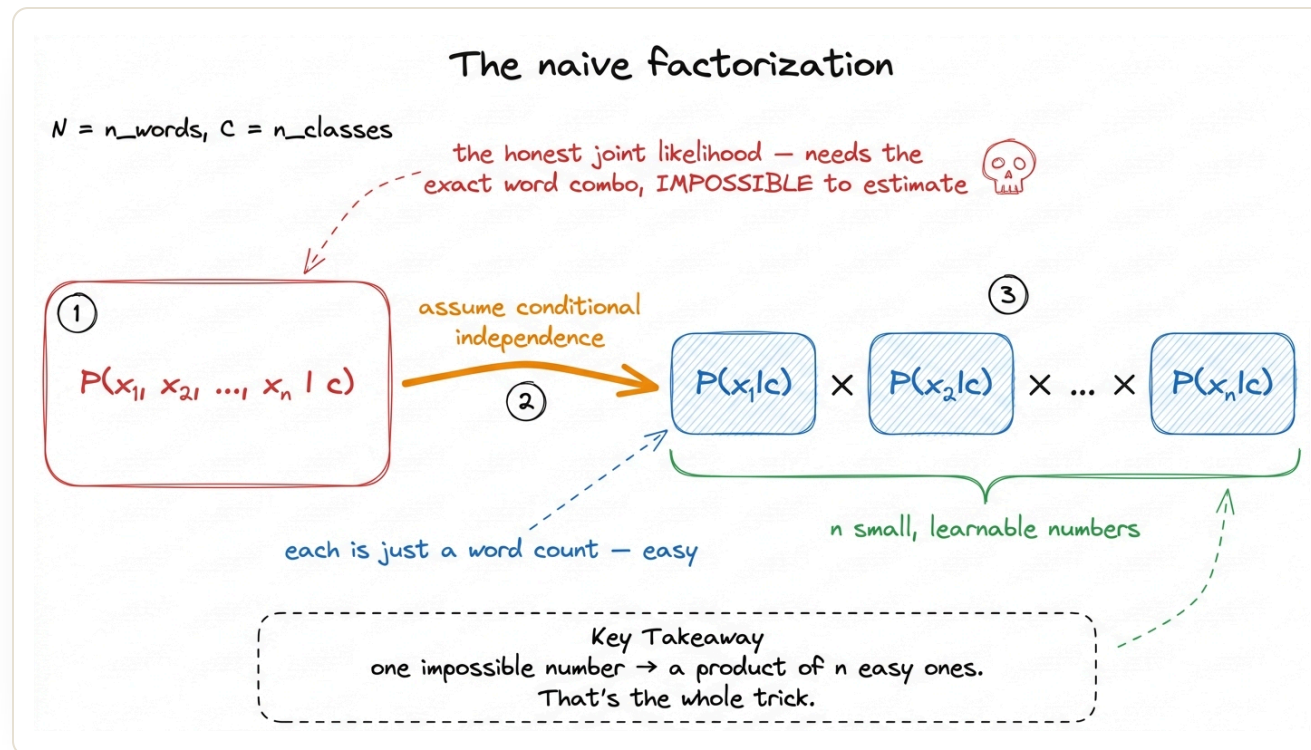
With one clue this is easy. The trouble starts the moment the evidence is a whole email — say the words $x_1, x_2, \dots, x_n$. Now the likelihood we need is $P(x_1, x_2, \dots, x_n \mid c)$: the probability of *this exact combination of words* given the class. To estimate that from data, we would need to have seen that exact combination before, many times, for every class. An email of twenty words drawn from a 10,000-word vocabulary has more possible combinations than there are atoms in your body. We will never have enough data. This is the wall every probabilistic classifier hits, and Naive-Bayes walks through it with one bold move.

The naive move, made concrete

The move is this: **assume the features are conditionally independent given the class**. Once we know the email is spam, knowing that "free" appeared tells us nothing extra about whether "prize" appeared. Under that assumption the impossible joint likelihood factors into a product of easy one-word likelihoods:

$$P(x_1, x_2, \dots, x_n | c) = P(x_1 | c) \cdot P(x_2 | c) \cdot \dots \cdot P(x_n | c)$$

Each term on the right is something we *can* estimate: just count how often word x_i shows up in the emails of class c . We traded one hopeless number for n easy ones, as the figure makes vivid.

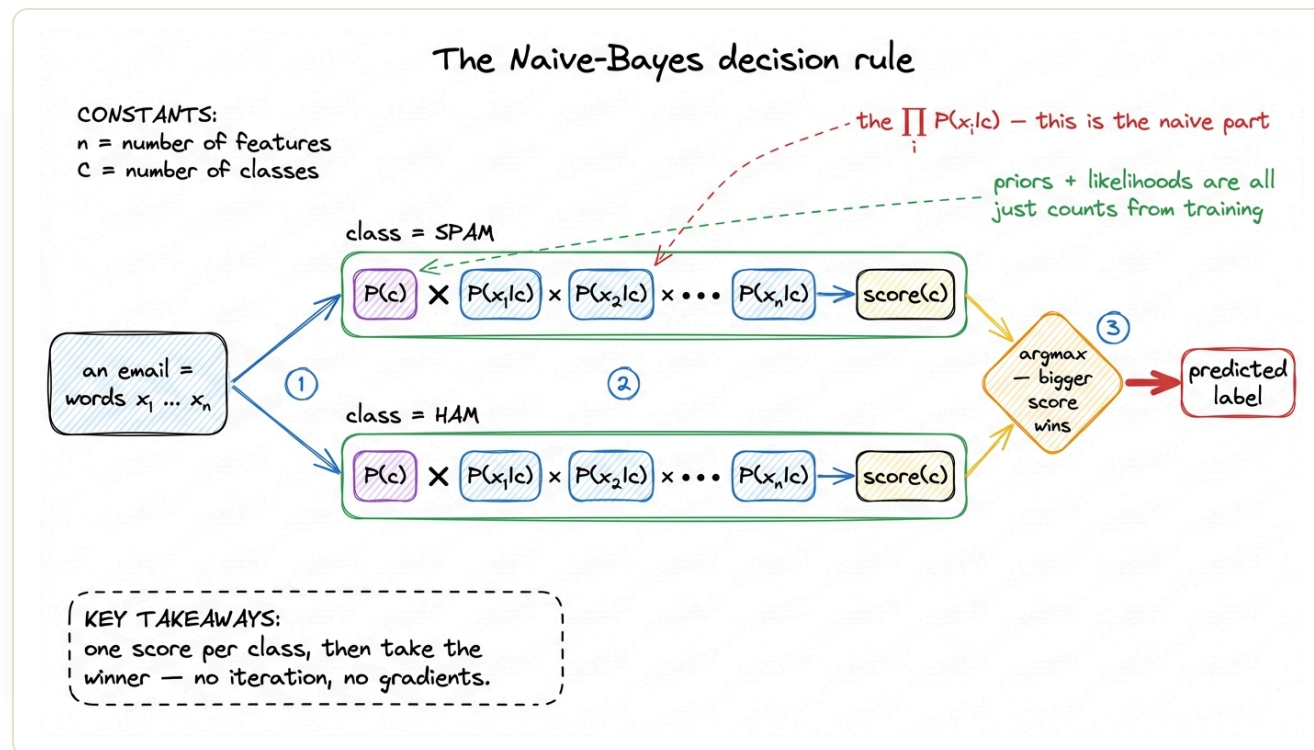


The naive assumption factors an unestimable joint likelihood into a product of countable per-word likelihoods.

Put it back into Bayes and drop the constant evidence term, and the classifier's entire decision rule is:

$$\text{predict} = \operatorname{argmax}_c P(c) \cdot \prod_i P(x_i|c)$$

Read it aloud: **pick the class that maximizes the prior times the product of per-word likelihoods.** That product over independent features is the "naive" heart of the method, and everything else is bookkeeping. The figure below lays the full rule out as a pipeline so you can see where every number enters.



The full classifier as a pipeline: score every class by prior times its product of word-likelihoods, then argmax to pick the label.

A worked spam example on the back of a napkin

Let me make this real with numbers small enough to check by hand.

Suppose we have trained on a tiny corpus and learned two things. First, the **priors**: $P(\text{spam}) = 0.4$ and $P(\text{ham}) = 0.6$ — most mail is legitimate.²

Second, from word counts, the per-word likelihoods for the three words in a new email — "free", "money", "lunch":

word	$P(\text{word} \mid \text{spam})$	$P(\text{word} \mid \text{ham})$
free	0.7	0.1
money	0.5	0.2
lunch	0.1	0.4

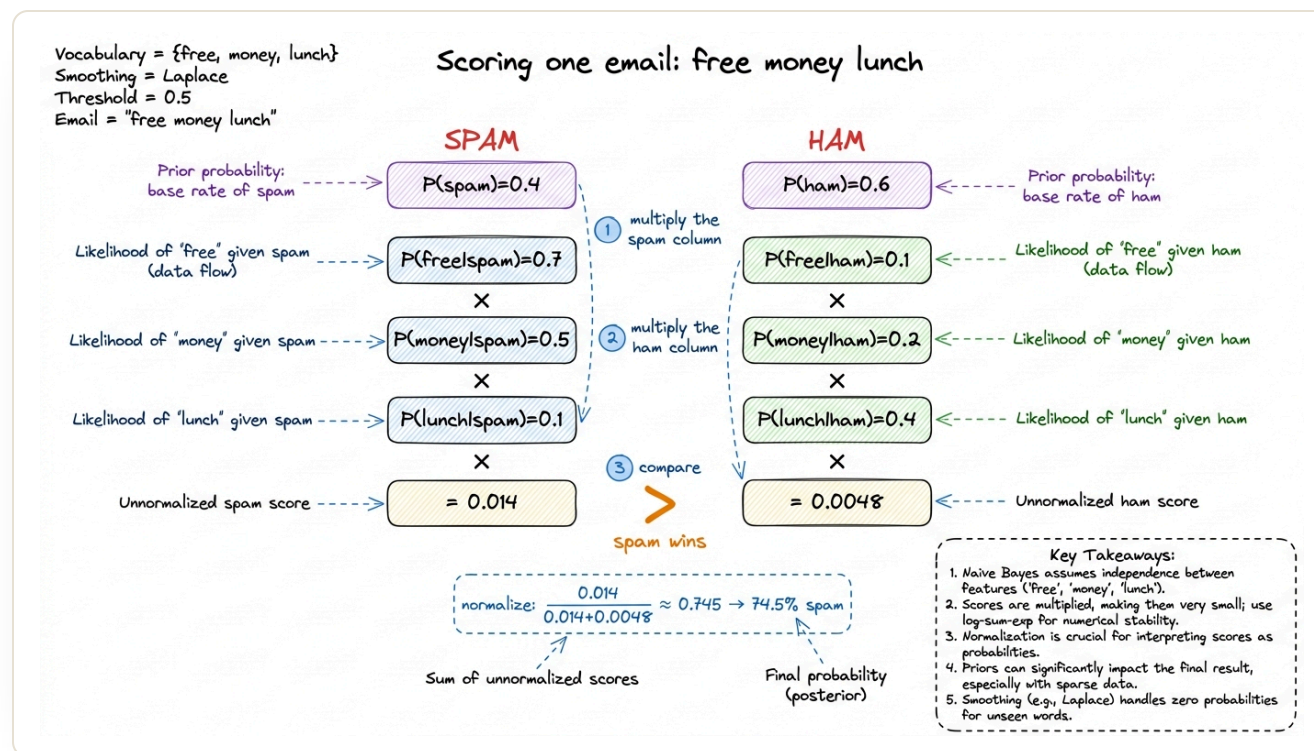
Now score the email "free money lunch" under each class. For **spam** we multiply the prior by the three spam-likelihoods:

$$\text{score}(\text{spam}) = 0.4 \times 0.7 \times 0.5 \times 0.1 = 0.014$$

For **ham** we do the same with the ham column:

$$\text{score}(\text{ham}) = 0.6 \times 0.1 \times 0.2 \times 0.4 = 0.0048$$

As the output shows, spam wins, $0.014 > 0.0048$, so the classifier labels the email spam. If we want an actual probability rather than a bare comparison, we can normalize by the sum: $P(\text{spam} \mid \text{email}) = 0.014 / (0.014 + 0.0048) \approx 0.745$. We compute about a 74.5% belief in spam, driven mostly by that large 0.7 likelihood on "free". Note that "lunch" pushed the other way, however, it is four times more likely in ham. This was not enough to overturn the other two words. This tug-of-war between features, one number at a time, is the fundamental mechanism of the model.



Multiply the prior by each word's likelihood down each class column, then compare — spam scores higher, giving a 74.5% posterior.

The pitfall: correlated words get counted twice

Here is where the naive assumption causes issues, and it often confuses practitioners the first time they encounter it. For example, imagine the words "free" and "prize" in spam. In real mail they almost always appear together, so they are effectively one signal serving two roles. Naive-Bayes does not know that, however. It multiplies $P(\text{free} \mid \text{spam})$ and $P(\text{prize} \mid \text{spam})$ as if they were two fresh, unrelated pieces of evidence, so the same underlying clue gets counted twice. Suppose each is 0.8 in spam and 0.1 in ham. The pair contributes a factor of $0.8 \times 0.8 = 0.64$ to the spam score against $0.1 \times 0.1 = 0.01$ to ham, which creates a 64-to-1 swing from what is essentially a single observation. The winner is still the correct one, but the *margin* is wildly inflated. This is exactly why the output probabilities pile up near 0.99 or 0.01. When we see a Naive-Bayes model report 99.9% confidence, we should not read it as a calibrated 99.9%. We can read it as a strong confidence where the magnitude is not to be trusted. As shown in the figure below, we can observe this double-count in action.

CONSTANTS: V = vocab size, count = word count in class

Two fixes every real Naive-Bayes needs

① (A) zero-probability trap

$$0.7 \times 0.5 \times 0.0 \times 0.9$$

$$= 0$$

→ one unseen word kills the class

FIX: Laplace smoothing $(\text{count}+1)/(\text{count}+V)$

→ no probability is ever 0

② (B) underflow

$$0.01 \times 0.02 \times 0.03 \times \dots \rightarrow 0.0 \text{ (float underflow)}$$

→ product of tiny → sum of negatives, same argmax

FIX: work in log space $\sum \log P(x_i|c)$

③ smooth so nothing is impossible; log so nothing underflows.

Because it multiplies correlated words as if independent, Naive-Bayes counts one clue twice and reports an over-confident, poorly calibrated probability.

Two problems the naive version hits, and their fixes

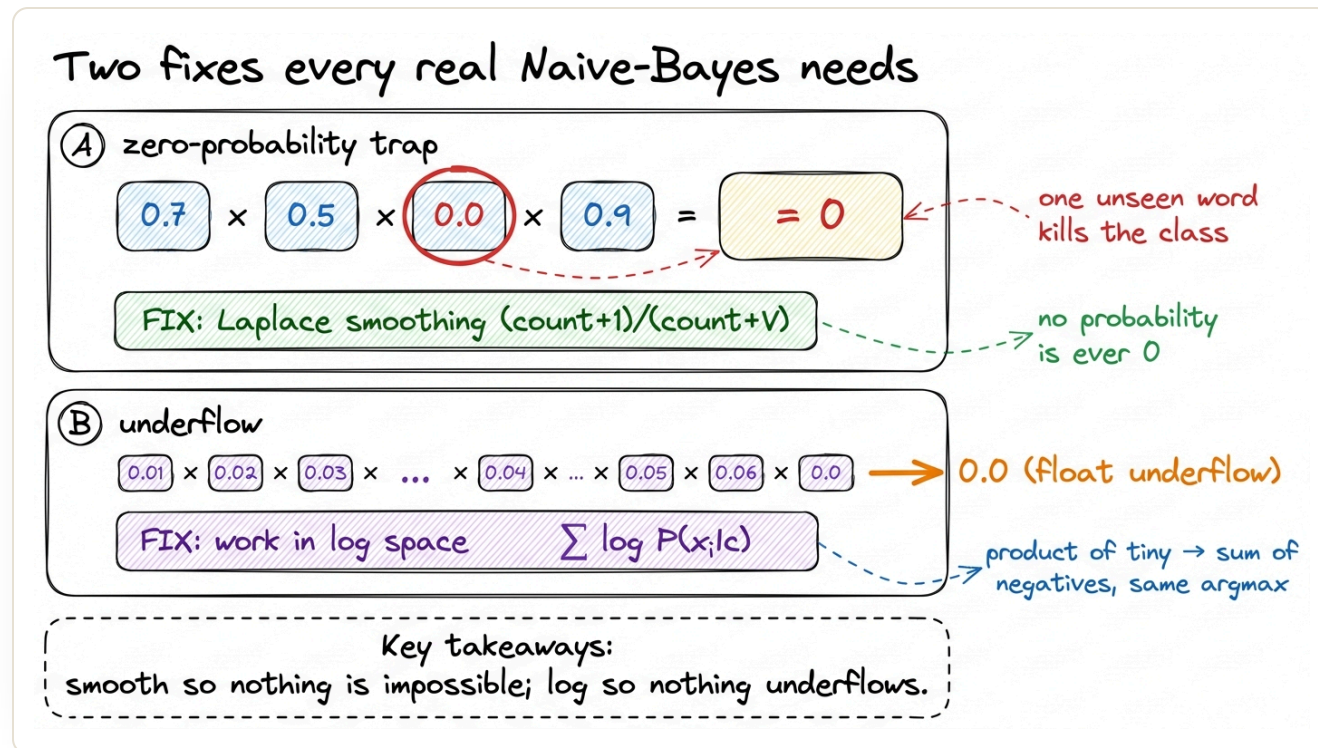
The previous calculations hide two failure modes that occur the moment we run this on real data, however, both of these issues have clean, standard fixes.

The zero-probability trap. Suppose the word "blockchain" never appeared in any ham email in our training set. Then $P(\text{blockchain} \mid \text{ham}) = 0$, and because we *multiply* the likelihoods, one zero drags the entire ham score to zero. A single unseen word vetoes the whole class, however, no matter what the other words say. The fix is **Laplace smoothing** (also called add-one smoothing), where we pretend every word was seen one extra time in every class. Instead of $\text{count}(\text{word}, c) / \text{count}(c)$, we compute $(\text{count}(\text{word}, c) + 1) / (\text{count}(c) + V)$, where V is the vocabulary size.³ Now no probability is ever exactly zero, and an unseen word merely nudges the score instead of completely eliminating the class probability.

Numerical underflow. If we multiply a few hundred probabilities that are each below 1, the product approaches zero faster than a floating-point number can represent. It silently becomes 0.0 , however, and every class ties. The standard fix is to work in **log space**. Because $\log$ is a monotonic function, the class that maximizes the product also maximizes the sum of logs, so the argmax operation remains unchanged:

$$\text{predict} = \operatorname{argmax}_c [\log P(c) + \sum_i \log P(x_i | c)]$$

A product of tiny numbers becomes a sum of manageable negative numbers, and the underflow issue vanishes. In every practical implementation we use the log form rather than the product form.



The two production fixes: Laplace smoothing stops a single unseen word from zeroing a class, and log-space turns a vanishing product into a stable sum.

Building it in a few lines

With both fixes implemented, the complete classifier is quite concise and straightforward to understand. For example, here it is with the estimator API we will discuss in detail in the [scikit-learn chapter](#), fitting on a handful of labeled sentences and predicting on a new one:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["free money now", "win a free prize",
         "lunch meeting at noon", "the project lunch is ready"]
labels = ["spam", "spam", "ham", "ham"]

X = CountVectorizer().fit_transform(texts)  # word-count matrix, shape (4, voc
clf = MultinomialNB(alpha=1.0).fit(X, labels)  # alpha=1.0 IS Laplace smoothin
```

`MultinomialNB` is the Naive-Bayes variant built for word counts, and `alpha=1.0` is exactly the add-one smoothing from our previous discussion.⁴ The `.fit()` call simply counts the words per class and stores the smoothed log-likelihoods. There is no gradient descent, no iteration, and no learning rate. This explains why Naive-Bayes trains in a single pass over the data, which often takes milliseconds where other models take minutes.

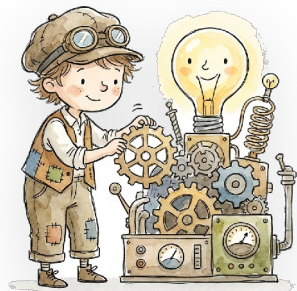
So why does a false assumption work so well?

Here is the resolution to the puzzle we opened with. The independence assumption is factually incorrect, however, because words are correlated. The *probabilities* Naive-Bayes outputs are consequently often badly calibrated, pushed toward a confident 0 or 1. Classification does not need calibrated probabilities, though. It only needs the **correct class to come out on top of the argmax**. We find that getting the ranking right is a far lower bar than getting the probabilities right. The correlated evidence tends to push the score in the *same direction* it would have anyway, just with the wrong magnitude. Therefore, we can conclude that **Naive-Bayes is often a poor estimator of probability but a surprisingly good picker of the winner**, and for classification the winner is all that matters.

That characteristic, plus a training cost of one pass and a memory cost of one table of word counts, is why it remains the standard baseline we reach for first. It is fast to build, hard to beat by a large margin, and clear about exactly what it computes.

Of course, stating that the model labeled this email spam is only half the story. We still need to ask how *often* it is right, and whether being right on

spam is the same as being right on ham. This is where the [next chapter on accuracy, precision, and recall](#) picks up, turning a pile of predictions into a reliable evaluation metric.



CHAPTER COMPLETE

Nicely done. You reached the end of this one.

NEXT CHAPTER Evaluating a Model: Accuracy, Precision, Recall



0

Did this chapter land for you?

[Mark chapter as done](#)

Rate this chapter ★ ★ ★ ★ ★

What did you think of this chapter?

A takeaway, a question, a moment that clicked...

Be kind and specific. 0/2000

Post

No notes yet — be the first to share what you took away.

[< Probability Distributio...](#)[◇ Back to book](#)[Evaluating a Model: Acc... >](#)